

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8"/>

<meta name="viewport" content="width=device-width,initial-scale=1.0"/>

<title>Thread Management System</title>

<style>

*,*::before,*::after{box-sizing:border-box;margin:0;padding:0;}

:root{

  --bg:#fff;--bg2:#f5f5f3;--bg3:#eeecea;

  --tx:#1a1a18;--tx2:#6b6a64;--tx3:#9c9a92;

  --bd:rgba(0,0,0,.12);--bd2:rgba(0,0,0,.22);

  --r:8px;--rl:12px;

  --ib:#E6F1FB;--it:#0C447C;--ibr:#B5D4F4;

  --ob:#EAF3DE;--ot:#27500A;--obr:#C0DD97;

  --wb:#FAEEDA;--wt:#633806;--wbr:#FAC775;

  --eb:#FCEBEB;--et:#501313;--ebr:#F7C1C1;

  --fs:system-ui,-apple-system,sans-serif;

  --fm:'Menlo','Consolas',monospace;

}

@media(prefers-color-scheme:dark){:root{

  --bg:#1e1e1c;--bg2:#2a2a27;--bg3:#333330;

  --tx:#e8e6de;--tx2:#9c9a92;--tx3:#6b6a64;

  --bd:rgba(255,255,255,.1);--bd2:rgba(255,255,255,.18);

  --ib:#042C53;--it:#B5D4F4;--ibr:#185FA5;

  --ob:#173404;--ot:#C0DD97;--obr:#3B6D11;

  --wb:#412402;--wt:#FAC775;--wbr:#854F0B;

  --eb:#501313;--et:#F7C1C1;--ebr:#A32D2D;

}}

body{font-family:var(--fs);background:var(--bg3);color:var(--tx);min-height:100vh;padding:1.25rem 1.5rem;}
```

```

h1{font-size:20px;font-weight:500;margin-bottom:3px;}

.sub{font-size:13px;color:var(--tx2);margin-bottom:1.1rem;}

.sec{font-size:11px;font-weight:500;color:var(--tx2);letter-spacing:.06em;text-
transform:uppercase;margin-bottom:8px;}

.panel{background:var(--bg);border:.5px solid var(--bd);border-radius:var(--rl);padding:13px
15px;margin-bottom:11px;}

/* scheduler row */

.srow{display:flex;gap:6px;margin-bottom:1rem;flex-wrap:wrap;align-items:center;}

.slbl{font-size:12px;color:var(--tx2);margin-right:2px;}

.sbtn{font-size:12px;padding:5px 12px;border:.5px solid var(--bd2);border-
radius:20px;background:var(--bg);color:var(--tx2);cursor:pointer;font-family:var(--fs);}

.sbtn:hover{background:var(--bg2);}

.sbtn.active{background:var(--ib);color:var(--it);border-color:var(--ibr);font-weight:500;}

.qwrap{display:flex;align-items:center;gap:5px;font-size:12px;color:var(--tx2);}

#qi{width:42px;padding:4px 6px;border:.5px solid var(--bd2);border-radius:var(--r);background:var(--
bg);color:var(--tx);font-size:12px;font-family:var(--fs);}

/* controls */

.ctrl{display:flex;gap:7px;flex-wrap:wrap;margin-bottom:1.1rem;align-items:center;}

.btn{font-size:13px;padding:7px 13px;border:.5px solid var(--bd2);border-radius:var(--
r);background:var(--bg);color:var(--tx);cursor:pointer;transition:background .12s,transform .1s;font-
family:var(--fs);}

.btn:hover{background:var(--bg2);}

.btn.active{transform:scale(.97);}

.btn.pri{background:var(--ib);color:var(--it);border-color:var(--ibr);}

.btn.ok{background:var(--ob);color:var(--ot);border-color:var(--obr);}

.btn.wrn{background:var(--wb);color:var(--wt);border-color:var(--wbr);}

.btn.err{background:var(--eb);color:var(--et);border-color:var(--ebr);}

.ml{margin-left:auto;}

.spwrap{display:flex;align-items:center;gap:6px;font-size:12px;color:var(--tx2);}

#spd{width:80px;}

```

```

/* stats */

.stats{display:grid;grid-template-columns:repeat(5,1fr);gap:9px;margin-bottom:1rem;}

.sc{background:var(--bg2);border-radius:var(--r);padding:10px 12px;}

.sl{font-size:11px;color:var(--tx2);margin-bottom:3px;}

.sv{font-size:20px;font-weight:500;}

.cg{color:#27500A;}.ca{color:#633806;}.cb{color:#0C447C;}

/* thread rows */

.tlist{display:flex;flex-direction:column;gap:7px;}

.trow{display:flex;align-items:center;gap:8px;padding:7px 11px;border-radius:var(--r);border:.5px
solid var(--bd);background:var(--bg2);position:relative;overflow:hidden;transition:border-color
.2s,opacity .3s;flex-wrap:wrap;}

.trow.running{border-color:var(--obr);}

.trow.waiting{border-color:var(--wbr);}

.trow.blocked{border-color:var(--ebr);}

.trow.terminated{opacity:.35;}

.tdot{width:9px;height:9px;border-radius:50%;flex-shrink:0;}

.dn{background:#888780;}.dr{background:#378ADD;}.dru{background:#639922;}

.dw{background:#BA7517;}.db{background:#A32D2D;}.dt{background:#533450;}

.tname{font-size:13px;font-weight:500;min-width:46px;}

.tstate{font-size:11px;padding:2px 8px;border-radius:20px;font-weight:500;min-width:74px;text-
align:center;}

.sn{background:var(--bg2);color:var(--tx2);border:.5px solid var(--bd2);}

.sr{background:var(--ib);color:var(--it);}

.sru{background:var(--ob);color:var(--ot);}

.sw{background:var(--wb);color:var(--wt);}

.sb{background:var(--eb);color:var(--et);}

.ste{background:#FBEAF0;color:#4B1528;}

.pbadge{font-size:10px;padding:2px 6px;border-radius:var(--r);background:var(--bg2);color:var(--
tx2);border:.5px solid var(--bd);}

.binfo{font-size:10px;color:var(--tx3);min-width:60px;}

.barw{flex:1;background:var(--bg);border:.5px solid var(--bd);border-
radius:4px;height:7px;overflow:hidden;min-width:60px;}

```

```

.bar{height:100%;border-radius:4px;transition:width .35s;}

.pfill{position:absolute;left:0;top:0;height:100%;background:var(--ob);opacity:.12;pointer-
events:none;transition:width .35s;}

/* resource badges & toggle buttons inside each row */

.res-area{display:flex;gap:4px;align-items:center;flex-shrink:0;flex-wrap:wrap;}

.mbtn{font-size:10px;padding:2px 7px;border:.5px solid var(--bd);border-
radius:4px;background:transparent;color:var(--tx2);cursor:pointer;font-family:var(--fs);white-
space:nowrap;}

.mbtn:hover{background:var(--bg2);color:var(--tx);}

.mbtn.m-on{background:var(--eb);color:var(--et);border-color:var(--ebr);}

.mbtn.s-on{background:var(--ib);color:var(--it);border-color:var(--ibr);}

.badge{font-size:10px;padding:2px 6px;border-radius:4px;font-weight:500;}

.bm{background:var(--eb);color:var(--et);border:.5px solid var(--ebr);}

.bs{background:var(--ib);color:var(--it);border:.5px solid var(--ibr);}

/* two-col */

.two{display:grid;grid-template-columns:1fr 1fr;gap:11px;margin-bottom:11px;}

.mbox{padding:9px 11px;border-radius:var(--r);border:.5px solid var(--bd);background:var(--
bg2);margin-bottom:8px;}

.mtitle{font-size:11px;font-weight:500;color:var(--tx2);margin-bottom:4px;}

.mlk{color:#A32D2D;font-weight:500;font-size:12px;}

.mul{color:#27500A;font-weight:500;font-size:12px;}

.semtr{display:flex;gap:4px;margin-top:5px;}

.semsl{width:18px;height:18px;border-radius:4px;border:.5px solid var(--bd2);background:var(--
bg);transition:background .2s;}

.semsl.on{background:#378ADD;border-color:#185FA5;}

/* queue/timeline */

.qrow{display:flex;gap:5px;flex-wrap:wrap;margin-top:5px;min-height:24px;align-items:center;}

.qchip{font-size:10px;padding:2px 8px;border-radius:20px;background:var(--ib);color:var(--
it);border:.5px solid var(--ibr);}

.qmt{font-size:11px;color:var(--tx3);font-style:italic;}

```

```

.sviz{display:flex;gap:3px;flex-wrap:wrap;margin-top:7px;min-height:24px;}

.tslot{width:22px;height:22px;border-radius:3px;font-size:9px;display:flex;align-items:center;justify-
content:center;font-weight:500;border:.5px solid transparent;}

/* gantt */

.ginner{display:flex;flex-direction:column;gap:5px;}

.grow{display:flex;align-items:center;gap:6px;}

.glbl{font-size:11px;color:var(--tx2);min-width:34px;text-align:right;}

.gtrack{flex:1;height:18px;background:var(--bg2);border-
radius:3px;position:relative;overflow:hidden;}

.gblk{position:absolute;top:0;height:100%;border-radius:2px;opacity:.85;}

.gthrow{display:flex;margin-left:40px;}

.gtick{font-size:9px;color:var(--tx3);flex:1;text-align:left;}

/* log */

.log{max-height:145px;overflow-y:auto;font-size:11px;font-family:var(--fm);color:var(--tx2);line-
height:1.8;padding:7px 9px;background:var(--bg2);border-radius:var(--r);}

.le{display:flex;gap:9px;}

.lt2{color:var(--tx3);flex-shrink:0;}

.lc{color:#185FA5;}.lr{color:#3B6D11;}.lw{color:#854F0B;}.lx{color:#A32D2D;}.ls{color:var(--tx);}

/* legend */

.leg{display:flex;gap:10px;flex-wrap:wrap;margin-bottom:.9rem;}

.li{display:flex;align-items:center;gap:5px;font-size:11px;color:var(--tx2);}

.ld{width:8px;height:8px;border-radius:50%;}

/* sched info */

.sinfo{font-size:12px;color:var(--tx2);padding:6px 10px;background:var(--bg2);border-radius:var(--
r);border:.5px solid var(--bd);margin-bottom:1rem;}

/* finish banner */

.fin{display:none;padding:10px 14px;border-radius:var(--r);background:var(--ob);color:var(--
ot);border:.5px solid var(--obr);font-size:13px;font-weight:500;margin-bottom:11px;}

```

```

.fin.show{display:block;}

/* hint box */

.hint{font-size:12px;color:var(--tx2);background:var(--ib);border:.5px solid var(--ibr);color:var(--it);border-radius:var(--r);padding:8px 12px;margin-bottom:11px;}

@keyframes pulse{0%,100%{opacity:1}50%{opacity:.45}}

.rp{animation:pulse 1s ease-in-out infinite;}

@media(max-width:620px){.stats{grid-template-columns:repeat(3,1fr);}.two{grid-template-columns:1fr;}.ml{margin-left:0;}}

</style>

</head>

<body>

<h1>Thread Management System</h1>

<p class="sub">Toggle mutex/semaphore need per thread — contention is automatic</p>

<!-- Scheduler -->

<div class="srow">

  <span class="sibl">Scheduler:</span>

  <button class="sbtn active" id="sb-rr" onclick="setSched('rr')">Round Robin</button>

  <button class="sbtn" id="sb-fcfs" onclick="setSched('fcfs')">FCFS</button>

  <button class="sbtn" id="sb-sjf" onclick="setSched('sjf')">SJF</button>

  <button class="sbtn" id="sb-prio" onclick="setSched('prio')">Priority</button>

  <button class="sbtn" id="sb-mlfq" onclick="setSched('mlfq')">MLFQ</button>

  <div class="qwrap" id="qwrap">

    <span>Quantum:</span><input type="number" id="qi" value="3" min="1" max="10"/><span>ticks</span>

  </div>

</div>

<div class="sinfo" id="sinfo"></div>

```

<!-- Controls -->

<div class="ctrl">

<button class="btn pri" onclick="createThread()">+ Create thread</button>

<button class="btn ok" id="btnrun" onclick="toggleRun()">▶ Run lifecycle</button>

<button class="btn" id="btnstep" onclick="stepOnce()">Step →</button>

<button class="btn wrn" onclick="releaseMutex()">Release mutex</button>

<button class="btn wrn" onclick="releaseSem()">Release semaphore</button>

<button class="btn err ml" onclick="resetAll()">Reset</button>

<div class="spwrap">

Speed:

<input type="range" id="spd" min="1" max="5" value="3" oninput="updateSpeed()"/>

Normal

</div>

</div>

<div class="hint">

💡 Toggle 🔒 Mutex or 🟦 Sem on any thread row to mark that thread as needing that resource.

When the simulation runs, threads that need a locked resource are blocked automatically.

Use Release mutex / Release semaphore to free resources and wake blocked/waiting threads.

</div>

<!-- Stats -->

<div class="stats">

<div class="sc"><div class="sl">Total</div><div class="sv" id="st">0</div></div>

<div class="sc"><div class="sl">Running</div><div class="sv cg" id="sr">0</div></div>

<div class="sc"><div class="sl">Waiting/Blocked</div><div class="sv ca" id="sw">0</div></div>

<div class="sc"><div class="sl">Terminated</div><div class="sv" id="sd">0</div></div>

<div class="sc"><div class="sl">CPU ticks</div><div class="sv cb" id="sc2">0</div></div>

</div>

```
<div class="fin" id="fin"></div>
```

```
<!-- Legend -->
```

```
<div class="leg">
```

```
  <div class="li"><div class="ld" style="background:#888780"></div>New</div>
```

```
  <div class="li"><div class="ld" style="background:#378ADD"></div>Ready</div>
```

```
  <div class="li"><div class="ld" style="background:#639922"></div>Running</div>
```

```
  <div class="li"><div class="ld" style="background:#BA7517"></div>Waiting (sem)</div>
```

```
  <div class="li"><div class="ld" style="background:#A32D2D"></div>Blocked (mutex)</div>
```

```
  <div class="li"><div class="ld" style="background:#533450"></div>Terminated</div>
```

```
</div>
```

```
<!-- Thread list -->
```

```
<div class="panel">
```

```
  <div class="sec">Thread list — click  or  to assign resource need</div>
```

```
  <div class="tlist" id="tlist">
```

```
    <div style="font-size:13px;color:var(--tx3);padding:6px 0">No threads yet. Click "+ Create thread".</div>
```

```
  </div>
```

```
</div>
```

```
<!-- Sync + Queue -->
```

```
<div class="two">
```

```
  <div class="panel" style="margin-bottom:0">
```

```
    <div class="sec">Synchronization state</div>
```

```
    <div class="mbox">
```

```
      <div class="mtitle">Mutex — shared resource lock</div>
```

```
      <div id="mxst"><span class="mul">UNLOCKED — available</span></div>
```

```
      <div style="font-size:11px;color:var(--tx3);margin-top:3px" id="mxinfo">No holder</div>
```

```
    </div>
```



```

<div class="mbox" style="margin-bottom:0">

  <div class="mtitle">Semaphore — capacity: 3 slots</div>

  <div class="semtr" id="semtr">

    <div class="semsl"></div><div class="semsl"></div><div class="semsl"></div>

  </div>

  <div style="font-size:11px;color:var(--tx3);margin-top:4px" id="seminfo">0 / 3 in use</div>

</div>

<div class="panel" style="margin-bottom:0">

  <div class="sec">Ready queue</div>

  <div class="qrow" id="rqueue"><div class="qmt">Queue empty</div></div>

  <div style="margin-top:12px">

    <div class="sec">CPU timeline (last 20 ticks)</div>

    <div class="sviz" id="sviz"><span style="font-size:11px;color:var(--tx3)">No history</span></div>

  </div>

</div>

</div>

<!-- Gantt -->

<div class="panel">

  <div class="sec">Gantt chart</div>

  <div id="gantt"><span style="font-size:12px;color:var(--tx3)">Run the simulation to see the Gantt
chart.</span></div>

</div>

<!-- Log -->

<div class="panel">

  <div class="sec">Event log</div>

  <div class="log" id="logp"></div>

</div>

```

```

<script>

const
COLORS=['#3B8BD4','#639922','#D85A30','#7F77DD','#1D9E75','#D4537E','#BA7517','#E24B4A'];

const PRIOS=['Low','Normal','High'];

const SPEEDS=[900,500,280,130,50];

const SLBLS=['Very slow','Slow','Normal','Fast','Turbo'];

const SDESC={
  rr: 'Round Robin — each thread gets a fixed time quantum in rotation.',
  fcfs:'FCFS — threads run in arrival order to completion.',
  sjf: 'SJF (preemptive) — always picks the thread with least remaining burst.',
  prio:'Priority (preemptive) — highest priority thread always runs first.',
  mlfq:'MLFQ — round-robin with quantum; threads that use full quantum go to back of queue.'
};

/* — State — */
let threads=[], nextId=1;

let mutexLocked=false, mutexHolder=null; // mutexHolder = thread NAME
let semHolders=new Set(); // set of thread IDs holding sem slots
let cpuTicks=0, cpuTimeline=[], tick=0;

let sched='rr', rrQueue=[], rrRem=0;

let timer=null, running=false;

let ganttData={};

/* — Scheduler — */
function setSched(s){
  sched=s;

  ['rr','fcfs','sjf','prio','mlfq'].forEach(k=>document.getElementById('sb-
'+k).classList.toggle('active',k===s));

  document.getElementById('qwrap').style.display=(s==='rr' || s==='mlfq')?'flex':'none';
  document.getElementById('sinfo').textContent=SDESC[s];

  threads.filter(t=>t.state==='running').forEach(t=>t.state='ready');

  rrQueue=[]; rrRem=0;

```

```

log(`Switched to ${s.toUpperCase()} scheduler`,`ls`);
render();
}
function getQ(){ return Math.max(1,+document.getElementById('qi').value | 3); }
function getSpd(){ return SPEEDS[+document.getElementById('spd').value-1]; }
function updateSpeed(){
    document.getElementById('splbl').textContent=SLBLS[+document.getElementById('spd').value-1];
    if(running){ clearInterval(timer); timer=setInterval(stepOnce,getSpd()); }
}

```

```

/* — Logging — */
function ts(){ return String(tick).padStart(4,'0'); }
function log(msg,cls='ls'){
    tick++;
    const p=document.getElementById('logp');
    const d=document.createElement('div'); d.className='le';
    d.innerHTML=`<span class="lt2">[${ts()}]</span><span class="${cls}">${msg}</span>`;
    p.appendChild(d); p.scrollTop=p.scrollHeight;
}

```

```

/* — Create thread — */
function createThread(){
    const id=nextId++;
    const burst=4+Math.floor(Math.random()*12);
    const prio=Math.floor(Math.random()*3);
    const color=COLORS[(id-1)%COLORS.length];
    threads.push({
        id, name:`T-${id}`, state:'ready', priority:prio, color,
        burst, remaining:burst, cpu:0, arrivalTick:cpuTicks, waitTime:0,
        needsMutex:false, // user toggles this per thread
        needsSem:false    // user toggles this per thread
    });
}

```

```

});

ganttData[id]=[];

log(`Created T-${id} | burst=${burst} | priority=${PRIOS[prio]}`, 'lc');

render();
}

/* — Toggle resource needs — called from each thread's row button — */
function toggleMutex(id){
  const t=threads.find(t=>t.id===id);
  if(!t || t.state==='terminated') return;
  t.needsMutex=!t.needsMutex;
  log(`${t.name}: mutex need → ${t.needsMutex?'ON (will auto-block if mutex taken)':'OFF'}`, 'ls');
  // Immediately apply contention if simulation is paused
  applyContention();
  render();
}

function toggleSem(id){
  const t=threads.find(t=>t.id===id);
  if(!t || t.state==='terminated') return;
  t.needsSem=!t.needsSem;
  log(`${t.name}: semaphore need → ${t.needsSem?'ON (will auto-wait if slots full)':'OFF'}`, 'ls');
  applyContention();
  render();
}

```

/* _____

applyContention — the core of the auto-blocking logic.

Called every tick AND when toggles change.

Rules:

1. If a thread needs mutex and mutex is FREE → give it the mutex

2. If a thread needs mutex and mutex is TAKEN (by someone else) → block it
3. If a thread needs sem and already holds a slot → do nothing
4. If a thread needs sem and slots available → give it a slot
5. If a thread needs sem and slots full → wait it
6. When a blocked thread's resource becomes free → wake it to ready

```

*/
function applyContention(){
  // Process every non-terminated thread
  for(const t of threads){
    if(t.state==='terminated') continue;

    /* --- Mutex --- */
    if(t.needsMutex){
      if(mutexHolder===t.name){
        // Already holds it — fine, keep running/ready
      } else if(!mutexLocked){
        // Free — give it to this thread
        mutexLocked=true; mutexHolder=t.name;
        log(`${t.name} acquired mutex → LOCKED`, 'ls');
      } else {
        // Taken by someone else — block if not already blocked/waiting
        if(t.state==='ready' || t.state==='running'){
          // If this thread was running, preempt it
          t.state='blocked';
          log(`${t.name} → BLOCKED (needs mutex, held by ${mutexHolder})`, 'lw');
        }
      }
    }
  }

  /* --- Semaphore --- */
  if(t.needsSem){

```

```

if(semHolders.has(t.id)){
  // Already holds a slot — fine
} else if(semHolders.size<3){
  // Slot available — give it
  semHolders.add(t.id);
  log(`${t.name} acquired semaphore slot (${semHolders.size}/3)`, 'ls');
} else {
  // Full — wait if not already blocked/waiting/terminated
  if(t.state==='ready' || t.state==='running'){
    t.state='waiting';
    log(`${t.name} → WAITING (needs semaphore, 3/3 full)`, 'lw');
  }
}
}
}

/* — Release resources (manual buttons) — */
function releaseMutex(){
  if(!mutexLocked){log('Mutex is already unlocked', 'lw');return;}
  const prev=mutexHolder;
  // Find the thread that held it and turn off its need so it doesn't re-acquire
  const holder=threads.find(t=>t.name===mutexHolder);
  if(holder) holder.needsMutex=false;
  mutexLocked=false; mutexHolder=null;
  log(`${prev} released mutex → UNLOCKED`, 'ls');
  // Wake all blocked threads — applyContention will re-block the ones that still need it
  // and give the mutex to the first one that needs it
  threads.filter(t=>t.state==='blocked').forEach(t=>{
    t.state='ready';
    if(!rrQueue.includes(t.id)) rrQueue.push(t.id);
  });
}

```

```

});

applyContention(); // will give mutex to first needy thread, block the rest

render();
}

function releaseSem(){
  // Release all slots (simplest UX — user can re-toggle needs)
  const holders=[...semHolders];
  if(!holders.length){log('No thread holds a semaphore slot','lw');return;}
  // Release one slot (the first holder)
  const id=holders[0];
  const holder=threads.find(t=>t.id===id);
  semHolders.delete(id);
  if(holder){
    holder.needsSem=false;
    log(` ${holder.name} released semaphore slot (${semHolders.size}/3)`,`ls`);
  }
  // Wake waiting threads
  threads.filter(t=>t.state==='waiting').forEach(t=>{
    t.state='ready';
    if(!rrQueue.includes(t.id)) rrQueue.push(t.id);
  });
  applyContention();
  render();
}

/* — Release on termination — */
function releaseOnTerminate(t){
  if(mutexHolder===t.name){
    mutexLocked=false; mutexHolder=null;
    log(` ${t.name} terminated → mutex UNLOCKED`,`ls`);
    // Wake blocked threads and let applyContention re-assign

```

```

    threads.filter(th=>th.state==='blocked').forEach(th=>{
        th.state='ready';
        if(!rrQueue.includes(th.id)) rrQueue.push(th.id);
    });
}

if(semHolders.has(t.id)){
    semHolders.delete(t.id);
    log(`${t.name} terminated → semaphore slot freed (${semHolders.size}/3)`, 'ls');
    threads.filter(th=>th.state==='waiting').forEach(th=>{
        th.state='ready';
        if(!rrQueue.includes(th.id)) rrQueue.push(th.id);
    });
}
}

/* — Scheduler — */
function pickNext(){
    const ready=threads.filter(t=>t.state==='ready');
    if(!ready.length) return null;
    if(sched==='fcfs') return ready.reduce((a,b)=>a.arrivalTick<=b.arrivalTick?a:b);
    if(sched==='sjf') return ready.reduce((a,b)=>a.remaining<=b.remaining?a:b);
    if(sched==='prio') return ready.reduce((a,b)=>b.priority>a.priority?b:a);
    if(sched==='rr' || sched==='mlfq'){
        rrQueue=rrQueue.filter(id=>threads.find(t=>t.id===id&&t.state==='ready'));
        ready.forEach(t=>{ if(!rrQueue.includes(t.id)) rrQueue.push(t.id); });
        if(!rrQueue.length) return null;
        return threads.find(t=>t.id===rrQueue[0]) || null;
    }
    return ready[0];
}

```



```

/* — One simulation tick — */
function stepOnce(){
  // 1. Apply contention first — blocks/waits any thread that needs a taken resource
  applyContention();

  // 2. Increment wait time for ready threads
  threads.filter(t=>t.state==='ready').forEach(t=>t.waitTime++);

  let cur=threads.find(t=>t.state==='running');

  // 3. Quantum expiry (RR/MLFQ)
  if(cur&&(sched==='rr' || sched==='mlfq')){
    rrRem--;
    if(rrRem<=0&&cur.remaining>0){
      cur.state='ready';
      rrQueue=rrQueue.filter(id=>id!==cur.id);
      rrQueue.push(cur.id);
      log(`${cur.name} quantum expired → back of queue`, 'lw');
      cur=null;
    }
  }

  // 4. Priority preemption
  if(cur&&sched==='prio'){
    const best=threads.filter(t=>t.state==='ready').sort((a,b)=>b.priority-a.priority)[0];
    if(best&&best.priority>cur.priority){
      cur.state='ready';
      log(`${cur.name} preempted by ${best.name}`, 'lw');
      cur=null;
    }
  }
}

```

```
// 5. SJF preemption
```

```
if(cur&&sched==='sjf'){  
  const sh=threads.filter(t=>t.state==='ready').sort((a,b)=>a.remaining-b.remaining)[0];  
  if(sh&&sh.remaining<cur.remaining){  
    cur.state='ready';  
    log(`${cur.name} preempted by ${sh.name} (shorter burst)`, 'lw');  
    cur=null;  
  }  
}
```

```
// 6. Pick new thread if none running
```

```
if(!cur){  
  cur=pickNext();  
  if(!cur){ checkDone(); render(); return; }  
  cur.state='running';  
  if(sched==='rr' | | sched==='mlfq'){  
    rrRem=getQ();  
    rrQueue=rrQueue.filter(id=>id!==cur.id);  
  }  
  log(`${cur.name} → RUNNING (${sched.toUpperCase()}, rem=${cur.remaining})`, 'lr');  
}
```

```
// 7. Execute one tick
```

```
cur.remaining--;  
cur.cpu++;  
cpuTicks++;
```

```
// Gantt
```

```
const gd=ganttData[cur.id];  
if(gd.length&&gd[gd.length-1].end===cpuTicks-1) gd[gd.length-1].end=cpuTicks;
```

```

else gd.push({start:cpuTicks-1,end:cpuTicks,color:cur.color});
cpuTimeline.push({name:cur.name,color:cur.color});
if(cpuTimeline.length>20) cpuTimeline.shift();

// 8. Completion check
if(cur.remaining<=0){
  cur.state='terminated';
  releaseOnTerminate(cur);
  applyContention(); // wake-and-re-assign after release
  rrQueue=rrQueue.filter(id=>id!==cur.id);
  const ta=cpuTicks-cur.arrivalTick;
  log(`${cur.name} DONE — cpu:${cur.cpu} wait:${cur.waitTime} turnaround:${ta}`, 'lx');
  checkDone();
}

render();
}

/* — checkDone — */
function checkDone(){
  if(!threads.length) return;
  if(threads.every(t=>t.state==='terminated')){
    stopRun();
    const aw=(threads.reduce((s,t)=>s+t.waitTime,0)/threads.length).toFixed(1);
    const at=(threads.reduce((s,t)=>s+(cpuTicks-t.arrivalTick),0)/threads.length).toFixed(1);
    const f=document.getElementById('fin');
    f.textContent=`All ${threads.length} threads done — CPU ticks: ${cpuTicks} | Avg wait: ${aw} | Avg
turnaround: ${at}`;
    f.classList.add('show');
    log(`Simulation complete | avg wait=${aw} | avg turnaround=${at}`, 'ls');
  }
}

```

```
}
```

```
/* — Auto-run — */
```

```
function toggleRun(){
```

```
  if(running) stopRun(); else startRun();
```

```
}
```

```
function startRun(){
```

```
  if(!threads.filter(t=>t.state!=='terminated').length){
```

```
    log('No active threads. Create threads first.','lw'); return;
```

```
  }
```

```
  document.getElementById('fin').classList.remove('show');
```

```
  running=true;
```

```
  document.getElementById('btnrun').textContent=' || Pause';
```

```
  document.getElementById('btnrun').className='btn wrn';
```

```
  timer=setInterval(stepOnce,getSpd());
```

```
}
```

```
function stopRun(){
```

```
  running=false; clearInterval(timer); timer=null;
```

```
  document.getElementById('btnrun').textContent='► Run lifecycle';
```

```
  document.getElementById('btnrun').className='btn ok';
```

```
}
```

```
/* — Quick run/pause per thread — */
```

```
function quickAction(id){
```

```
  const t=threads.find(t=>t.id===id);
```

```
  if(!t || t.state==='terminated') return;
```

```
  if(t.state==='running'){
```

```
    t.state='ready';
```

```
    if(sched==='rr' || sched==='mlfq') rrQueue.push(id);
```

```
    log(`${t.name} manually paused → READY`, 'lw');
```

```
  } else if(t.state==='ready'){
```

```

    threads.filter(tt=>tt.state==='running').forEach(tt=>{
        tt.state='ready';
        if(sched==='rr' || sched==='mlfq') rrQueue.push(tt.id);
    });
    t.state='running'; rrRem=getQ();
    if(sched==='rr' || sched==='mlfq') rrQueue=rrQueue.filter(i=>i!==id);
    log(` ${t.name} manually resumed → RUNNING`, 'lr');
}
render();
}

```

```

/* — Reset — */
function resetAll(){
    stopRun();
    threads=[]; nextId=1; mutexLocked=false; mutexHolder=null;
    semHolders=new Set(); cpuTicks=0; cpuTimeline=[]; tick=0;
    rrQueue=[]; rrRem=0; ganttData={};
    document.getElementById('logp').innerHTML="";
    document.getElementById('fin').classList.remove('show');
    log('System reset', 'ls');
    render();
}

```

```

/* — Helpers — */
function
dc(s){return{new:'dn',ready:'dr',running:'dru',waiting:'dw',blocked:'db',terminated:'dt'}[s] || 'dn';}
function
sc(s){return{new:'sn',ready:'sr',running:'sru',waiting:'sw',blocked:'sb',terminated:'ste'}[s] || 'sn';}

```

```

/* — Render — */
function render(){
    document.getElementById('st').textContent=threads.length;
}

```

```

document.getElementById('sr').textContent=threads.filter(t=>t.state==='running').length;

document.getElementById('sw').textContent=threads.filter(t=>['waiting','blocked'].includes(t.state))
.length;

document.getElementById('sd').textContent=threads.filter(t=>t.state==='terminated').length;

document.getElementById('sc2').textContent=cpuTicks;


// Thread list
const list=document.getElementById('tlist');
list.innerHTML=threads.length
? threads.map(t=>{
    const pct=Math.round(((t.burst-t.remaining)/t.burst)*100);
    const hm=muxHolder===t.name;
    const hs=semHolders.has(t.id);
    return `<div class="trow ${t.state}">
        <div class="pfill" style="width:${pct}%"></div>
        <div class="tdot ${dc(t.state)}${t.state==='running'?' rp':''}"></div>
        <span class="tname">${t.name}</span>
        <span class="tstate ${sc(t.state)}">${t.state.toUpperCase()}</span>
        <span class="pbadge">${PRIOS[t.priority]}</span>
        <span class="binfo">rem:${t.remaining}/${t.burst}</span>
        <div class="barw"><div class="bar" style="width:${pct}%;background:${t.color}"></div></div>
        <span style="font-size:10px;color:var(--tx3);min-width:52px">cpu:${t.cpu}
w:${t.waitTime}</span>
        ${t.state!=='terminated'?
        <div class="res-area">
            <button class="mbtn"
onclick="quickAction(${t.id})">${t.state==='running'?'Pause':'Run'}</button>
            <button class="mbtn ${t.needsMutex?'m-on':''}" onclick="toggleMutex(${t.id})"
            title="${t.needsMutex?'Turn OFF mutex need for this thread':'Turn ON: this thread needs the
mutex'}">
                🗝️ Mutex ${t.needsMutex?'ON':'OFF'}
            </button>

```

```

    <button class="mbtn ${t.needsSem?'s-on':''}" onclick="toggleSem(${t.id})"
        title="${t.needsSem?'Turn OFF semaphore need for this thread':'Turn ON: this thread needs
a semaphore slot'}">
        ● Sem ${t.needsSem?'ON':'OFF'}
    </button>

    ${hm?`<span class="badge bm">holds mutex</span>`:''}

    ${hs?`<span class="badge bs">holds sem</span>`:''}

</div>`:''}

</div>;

}).join("")

: '<div style="font-size:13px;color:var(--tx3);padding:6px 0">No threads. Click "+ Create
thread".</div>';

```

// Mutex panel

```

document.getElementById('mxst').innerHTML=mutexLocked
?`<span class="mlk">LOCKED — held by ${mutexHolder}</span>`
: '<span class="mul">UNLOCKED — available</span>';

const blk=threads.filter(t=>t.state==='blocked');

document.getElementById('mxinfo').textContent=blk.length
?`${blk.length} thread(s) blocked: ${blk.map(t=>t.name).join(', ')}`
: 'No threads blocked';

```

// Semaphore panel

```

document.querySelectorAll('#semtr
.semstl').forEach((s,i)=>s.classList.toggle('on',i<semHolders.size));

const snames=[...semHolders].map(id=>{const t=threads.find(t=>t.id===id);return t?t.name:'?'});

document.getElementById('seminfo').textContent=`${semHolders.size} / 3 in use`+
(snames.length?' — held by: '+snames.join(', '):'');

```

// Ready queue

```

let rq=threads.filter(t=>t.state==='ready');

if(sched==='fcfs') rq.sort((a,b)=>a.arrivalTick-b.arrivalTick);

```

```

else if(sched==='sjf') rq.sort((a,b)=>a.remaining-b.remaining);
else if(sched==='prio') rq.sort((a,b)=>b.priority-a.priority);
else rq.sort((a,b)=>rrQueue.indexOf(a.id)-rrQueue.indexOf(b.id));
document.getElementById('rqueue').innerHTML=rq.length

?rq.map(t=><span class="qchip" style="border-
color:${t.color};background:${t.color}18;color:${t.color}">${t.name}${PRIOS[t.priority][0]}</span>`)
.join("")

:'<div class="qmt">Queue empty</div>';

// CPU timeline
document.getElementById('sviz').innerHTML=cpuTimeline.length

?cpuTimeline.map(c=><div class="tslot" style="background:${c.color}22;border-
color:${c.color}66;color:${c.color}">${c.name.replace('T-', '')}</div>`).join("")

:'<span style="font-size:11px;color:var(--tx3)">No history</span>';

renderGantt();
}

function renderGantt(){
  const wrap=document.getElementById('gantt');
  const ids=Object.keys(ganttData).map(Number).filter(id=>ganttData[id].length>0);
  if(!ids.length || !cpuTicks){
    wrap.innerHTML='<span style="font-size:12px;color:var(--tx3)">Run the simulation to see the
Gantt chart.</span>';
    return;
  }
  const maxT=cpuTicks;
  let html='<div class="ginner">';
  ids.forEach(id=>{
    const t=threads.find(th=>th.id===id); if(!t) return;
    html+=`<div class="grow"><span class="gbl">${t.name}</span><div class="gtrack">`;
    ganttData[id].forEach(b=>{

```



```

    const l=(b.start/maxT)*100;

    const w=Math.max(((b.end-b.start)/maxT)*100,0.5);

    html+=`<div class="gblk"
style="left:${l.toFixed(2)}%;width:${w.toFixed(2)}%;background:${b.color}" title="${t.name}: ticks
${b.start}–${b.end}"></div>`;

    });

    html+=`</div></div>`;

});

const steps=Math.min(maxT,10);

html+=`<div class="gtrow">`;

for(let i=0;i<=steps;i++){

    const v=Math.round((i/steps)*maxT);

    html+=`<div class="gtick" style="flex:${i===0?'0 0 auto':'1'}">${v}</div>`;

}

html+=`</div></div>`;

wrap.innerHTML=html;

}

// Init

setSched('rr');

render();

</script>

</body>

</html>

```